

Посібник з навчання

Припасуйте параметри без міток із телеметрії та згенеруйте розгортвану мережу — історія відмов не потрібна

Покроковий огляд навчальної частини: що припасовує тренер, як його запустити, як підключити власні дані історіана, як перевірити результат і де саме навчання передає естафету розгортанню. Python тут лише для початкового припасування; середовище виконання — Java.

Документ

Посібник з навчання моделі

Продукт

Jneopallium Self-Supervised Maintenance Guardian

Тренер

scripts/demo-self-supervised-maintenance/
train_ss_maintenance_model.py

Використано міток

0 (самокероване)

Вихід

Розгортувана збірка Jneopallium (дескриптор + контекст
+ шари)

Автор

Дмитро Раковський — Харків, Україна

Дата

2 липня 2026

Ліцензія

BSD 3-Clause

Зміст

1. Чого ви досягнете
2. Де закінчується навчання і починається розгортання
3. Передумови
4. Крок 1 — запустити тренер
5. Крок 2 — що він припасував (параметри)
6. Крок 3 — збірка, яку він створив
7. Крок 4 — підключити власні дані
8. Крок 5 — перевірити тестами
9. Крок 6 — передача до розгортання
10. Перенавчання та дрейф
11. Усунення несправностей
12. Додаток — шпаргалка навчання

1 Чого ви досягнете

Наприкінці ви матимете набір припасованих параметрів без міток — стандартизацію за режимами, ваги міжсенсорного відновлення та перцентилі базового рівня здоров'я — записаних у повну розгортвану збірку Jneopallium, з якої середовище Java завантажується напряму. Мітки відмов не використовуються на жодному етапі.

Стеля безпеки стосується й навчання

Ніщо, що створює тренер, не може керувати пристроєм. Отримана модель є рекомендаційною за побудовою (SsMaintConfig забороняє це вимкнути), а тяжкість калібрується за здоровою історією — ніколи за розміченими відмовами, бо їх немає.

2 Де закінчується навчання і починається розгортання

Фаза	Що відбувається	Документ
Навчання (цей посібник)	Припасувати стандартизацію, ваги, базові рівні; згенерувати збірку	Посібник з навчання
Розгортання	Завантажити збірку, підключити телеметрію + зворотний зв'язок, запустити скорер	Посібник з розгортання

Естафета — це один каталог: `worker/src/main/resources/model/self-supervised-maintenance/`.
Навчання **записує** його; розгортання **читає** його.

3 Передумови

- Python 3.9+ (лише стандартна бібліотека — `pip install` не потрібен).
- JDK 17 і Maven, якщо ви також хочете запускати Java-тести середовища виконання.
- Репозиторій клоновано; команди нижче виконуються з кореня репозиторію.

4 Крок 1 — запустити тренер

```
cd scripts/demo-self-supervised-maintenance
python train_ss_maintenance_model.py
```

Очікуваний вивід (значення детерміновані для наявного синтетичного корпусу):

```
Self-Supervised Maintenance – initial (label-free) training

Trained on 6 assets x 2000 ticks, labels used: 0
Sensors                : 8
Health baseline mean   : 0.7781
Health baseline p99    : 2.0808
Health baseline p999   : 2.7544
Neurons in bundle      : 4
```

run_all.sh (або run_all.ps1) запускає тренер, а потім Python-тести за один прохід.

5 Крок 2 — що він припасував (параметри)

Усі три групи параметрів є самокерованими — припасованими лише з телеметрії, на довіреному здоровому вікні:

- **Стандартизація за режимами** — середнє й стандартне відхилення кожного сенсора в кожному режимі, щоб оцінювати зчитування відносно навантаження.
- **Ваги міжсенсорного відновлення** — для кожного сенсора модель гребеневої регресії, що передбачає його за іншими сенсорами. Це ядро без міток: ціль кожного припасування — інший сенсор, ніколи не мітка відмови.
- **Перцентилі базового рівня здоров'я** — середнє, p95, p99 і p999 помилки відновлення на здоровому вікні, для калібрування тяжкості у власних одиницях активу.

Чому гребенева регресія

Невеликий гребневий (L2) штраф тримає міжсенсорні моделі стабільними, коли сенсори корельовані — а на пов'язаній установці вони завжди такі. Розв'язувач — звичайне розв'язання нормальних рівнянь на чистому Python; сторонній пакет лінійної алгебри не потрібен.

6 Крок 3 — збірка, яку він створив

Файл	Призначення
model-descriptor.json	Маніфест: шари, сигнали, джерела даних, labelFree=true, labelsUsed=0
production-context.json	ІContext для запускача Entry (класи нейронів, входи, аргументи)
layer-0-input.json	Дескриптор прийому телеметрії + порядок сенсорів
layer-1-selfsupervisedhealth.json	Нейрон відновлення + припасовані ваги та стандартизація
layer-2-fusion.json	Нейрон гіпотези + базові перцентилі + константи поєднання
layer-3-onlinelearning.json	Нейрон адаптації + межі/обмеження частоти/заморожування
layer-4-advisorygate.json	Шлюз лише для читання + початкові пороги + вікно дедуплікації
fitted-model.json	Сирі припасовані параметри для аудиту / довідки

Файли шарів несуть параметри як поля нейронів, тож файли, що фіксують ваше припасування, — це файли, які прив'язує середовище виконання. Це віддзеркалює компонування збірки промислової моделі.

7 Крок 4 — підключити власні дані

Для реального майданчика замініть синтетичний генератор експортом свого історіана. Тренер читає список рядків по тактах через `synth_telemetry.generate`; спрямуйте його на свої дані. Кожен канонічний рядок має містити:

- Ті самі **назви сенсорів**, яких очікує модель (або відредагуйте `SENSORS` під свій набір тегів).
- Цілочисельний стовпець **режиму** (робоча точка / смуга навантаження). Якщо його немає — виведіть його зі смуги навантаження чи потужності.
- **Без міток**. Вікно навчання має бути **переважно здоровим** — це єдине важливе припущення. Якщо актив уже був несправним у цьому вікні, виключіть цей період, щоб він не отруїв «норму».

Базові рівні парку для холодного старту

Якщо в активу замало історії, щоб визначити власну норму, припасовуйте за спорідненими активами того ж режиму. Модель повідомляє зсув домену й невизначеність, тож незнайомий актив читається як «спостерігаю», а не робить упевнені заяви.

8 Крок 5 — перевірити тестами

```
cd scripts/demo-self-supervised-maintenance
python -m unittest tests.test_ss_maintenance -v
```

Python-тести підтверджують, що тренер створив надійну модель:

- перцентилі базового рівня правильно впорядковані ($\text{mean} < p95 < p99 < p999$);
- усі ваги міжсенсорного відновлення скінченні;
- здорові кадри рідко перевищують власний $p99$ активу (частка зайвих спрацювань нижче 5%);
- **кожне впроваджене сімейство несправностей відокремлюється від здорового базового рівня без міток** (помилка відновлення понад $2 \times p999$), і домінантний залишок вказує на правильну групу сенсорів;
- збірка записана й валідна (`labelFree=true`, `labelsUsed=0`, `safetyMode=ADVISORY`).

Поведінку середовища виконання — накопичення свідчень, адаптацію зворотного зв'язку, шлюз — перевіряє окремо Java-набір `SelfSupervisedMaintenanceModuleTest` (14 тестів). Запустіть його через `mvn -pl worker -Dtest=SelfSupervisedMaintenanceModuleTest test`.

9 Крок 6 — передача до розгортання

Навчання завершено, коли каталог збірки записаний і тести проходять. Далі за справу береться посібник з розгортання: він завантажує ту саму збірку запуском `Entry`, підключає вашу телеметрію OPC UA / MQTT до `AssetTelemetrySignal`, а зворотний зв'язок операторів до `OperatorFeedbackSignal`, і починає видавати рекомендації.

10 Перенавчання та дрейф

Перепасовуйте базові рівні, коли установка справді змінюється (новий продукт, капремонт, сезонний зсув) — а не на кожному сплеску. Щоденну адаптацію виконує під час роботи цикл зворотного зв'язку, якому не потрібні перенавчання чи переустановка. Практичний ритм: перепасовувати

стандартизацію й базові рівні на ковзному здоровому вікні щомісяця або після відомої зміни, а решту хай робить онлайн-цикл.

11 Усунення несправностей

Симптом	Ймовірна причина	Виправлення
Здорові активи часто сигналять	Вікно навчання не було здоровим	Виключити несправні періоди
Несправність не відокремлюється	Слабкий зв'язок / відсутній канал	Додати пов'язаний канал
Усе читається як невизначене	Дані не схожі на навчальні (зсув)	Перепасувати на типових даних
Перцентилі виглядають хибними	Замало тактів на режим	Дати кожному режиму досить вибірки

12 Додаток — шпаргалка навчання

```
# припасувати + згенерувати збірку
python scripts/demo-self-supervised-maintenance/train_ss_maintenance_model.py

# python-тести (тренер + відокремлення без міток)
python -m unittest tests.test_ss_maintenance

# java-тести середовища виконання
mvn -pl worker -Dtest=SelfSupervisedMaintenanceModuleTest test
```

Навчання тут означає припасування «норми» з телеметрії та генерацію розгортуваної мережі — з нульовою міткою. Усе про конкретну відмову вивчається пізніше, під час роботи, зі зворотного зв'язку операторів.